

# UIL COMPUTER SCIENCE WRITTEN TEST

# 2026 DISTRICT

**MARCH 2026**

## General Directions (Please read carefully!)

---

1. DO NOT OPEN THE EXAM UNTIL TOLD TO DO SO.
2. There are 40 questions on this contest exam. You will have 45 minutes to complete this contest.
3. All answers must be legibly written on the answer sheet provided. Indicate your answers in the appropriate blanks provided on the answer sheet. Clean erasures are necessary for accurate grading.
4. You may write on the test packet or any additional scratch paper provided by the contest director, but NOT on the answer sheet, which is reserved for answers only.
5. All questions have ONE and only ONE correct answer. There is a 2-point penalty for all incorrect answers.
6. Tests may not be turned in until 45 minutes have elapsed. If you finish the test before the end of the allotted time, remain at your seat and retain your test until told to do otherwise. You may use this time to check your answers.
7. If you are in the process of actually writing an answer when the signal to stop is given, you may finish writing that answer.
8. All provided code segments are intended to be syntactically correct, unless otherwise stated. You may also assume that any undefined variables are defined as used.
9. A reference to many commonly used Java classes is provided with the test, and you may use this reference sheet during the contest. AFTER THE CONTEST BEGINS, you may detach the reference sheet from the test booklet if you wish.
10. Assume that any necessary import statements for standard Java SE packages and classes (e.g., `java.util`, `System`, etc.) are included in any programs or code segments that refer to methods from these classes and packages.
11. NO CALCULATORS of any kind may be used during this contest.

## Scoring

---

1. Correct answers will receive **6 points**.
2. Incorrect answers will lose **2 points**.
3. Unanswered questions will neither receive nor lose any points.
4. In the event of a tie, the student with the highest percentage of attempted questions correct shall win the tie.

# STANDARD CLASSES AND INTERFACES – SUPPLEMENTAL REFERENCE

## package java.lang

```

class Object
    boolean equals(Object anotherObject)
    String toString()
    int hashCode()

interface Comparable<T>
    int compareTo(T anotherObject)
        Returns a value < 0 if this is less than anotherObject.
        Returns a value = 0 if this is equal to anotherObject.
        Returns a value > 0 if this is greater than anotherObject.

class Integer implements Comparable<Integer>
    Integer(int value)
    int intValue()
    boolean equals(Object anotherObject)
    String toString()
    String toString(int i, int radix)
    int compareTo(Integer anotherInteger)
    static int parseInt(String s)

class Double implements Comparable<Double>
    Double(double value)
    double doubleValue()
    boolean equals(Object anotherObject)
    String toString()
    int compareTo(Double anotherDouble)
    static double parseDouble(String s)

class String implements Comparable<String>
    int compareTo(String anotherString)
    boolean equals(Object anotherObject)
    int length()
    String substring(int begin)
        Returns substring(begin, length()).
    String substring(int begin, int end)
        Returns the substring from index begin through index (end - 1).
    int indexOf(String str)
        Returns the index within this string of the first occurrence of str. Returns
        -1 if str is not found.
    int indexOf(String str, int fromIndex)
        Returns the index within this string of the first occurrence of str, starting
        the search at fromIndex. Returns -1 if str is not found.
    int indexOf(int ch)
    int indexOf(int ch, int fromIndex)
    char charAt(int index)
    String toLowerCase()
    String toUpperCase()
    String[] split(String regex)
    boolean matches(String regex)
    String replaceAll(String regex, String str)

class Character
    static boolean isDigit(char ch)
    static boolean isLetter(char ch)
    static boolean isLetterOrDigit(char ch)
    static boolean isLowerCase(char ch)
    static boolean isUpperCase(char ch)
    static char toUpperCase(char ch)
    static char toLowerCase(char ch)

class Math
    static int abs(int a)
    static double abs(double a)
    static double pow(double base, double exponent)
    static double sqrt(double a)
    static double ceil(double a)
    static double floor(double a)
    static double min(double a, double b)
    static double max(double a, double b)
    static int min(int a, int b)
    static int max(int a, int b)
    static long round(double a)
    static double random()
        Returns a double greater than or equal to 0.0 and less than 1.0.
    
```

## package java.util

```

interface List<E>
class ArrayList<E> implements List<E>
    boolean add(E item)
    int size()
    Iterator<E> iterator()
    ListIterator<E> listIterator()
    E get(int index)
    E set(int index, E item)
    void add(int index, E item)
    E remove(int index)

class LinkedList<E> implements List<E>, Queue<E>
    void addFirst(E item)
    void addLast(E item)
    E getFirst()
    E getLast()
    E removeFirst()
    E removeLast()

class Stack<E>
    boolean isEmpty()
    E peek()
    E pop()
    E push(E item)

interface Queue<E>
class PriorityQueue<E>
    boolean add(E item)
    boolean isEmpty()
    E peek()
    E remove()

interface Set<E>
class HashSet<E> implements Set<E>
class TreeSet<E> implements Set<E>
    boolean add(E item)
    boolean contains(Object item)
    boolean remove(Object item)
    int size()
    Iterator<E> iterator()
    boolean addAll(Collection<? extends E> c)
    boolean removeAll(Collection<?> c)
    boolean retainAll(Collection<?> c)

interface Map<K,V>
class HashMap<K,V> implements Map<K,V>
class TreeMap<K,V> implements Map<K,V>
    Object put(K key, V value)
    V get(Object key)
    boolean containsKey(Object key)
    int size()
    Set<K> keySet()
    Set<Map.Entry<K, V>> entrySet()

interface Iterator<E>
    boolean hasNext()
    E next()
    void remove()

interface ListIterator<E> extends Iterator<E>
    void add(E item)
    void set(E item)

class Scanner
    Scanner(InputStream source)
    Scanner(String str)
    boolean hasNext()
    boolean hasNextInt()
    boolean hasNextDouble()
    String next()
    int nextInt()
    double nextDouble()
    String nextLine()
    Scanner useDelimiter(String regex)
    
```

# STANDARD CLASSES AND INTERFACES – SUPPLEMENTAL REFERENCE

## Package `java.util.function`

```
Interface BiConsumer<T,U>
    void accept(T t, U u)

Interface BiFunction<T,U,R>
    R apply(T t, U u)

Interface BiPredicate<T,U>
    boolean test(T t, U u)

Interface Consumer<T>
    void accept(T t)

Interface Function<T,R>
    R apply(T t)

Interface Predicate<T>
    boolean test(T t)

Interface Supplier<T>
    T get()
```

# UIL COMPUTER SCIENCE WRITTEN TEST – 2026 DISTRICT

**Note:** Correct responses are based on **Java SE Development Kit 22 (JDK 22)** from Oracle, Inc. All provided code segments are intended to be syntactically correct, unless otherwise stated (e.g., "error" is an answer choice) and any necessary Java SE 22 Standard Packages have been imported. Ignore any typographical errors and assume any undefined variables are defined as used. **For all output statements, assume that the System class has been statically imported using: `import static java.lang.System.*;`**

## Question 1

Which of the following is equivalent to the expression  $456_8 + 711_8$ ?

- A)  $745_{10}$                       B)  $744_8$                       C)  $2E7_{16}$                       D)  $1011100111_2$                       E)  $1367_8$

## Question 2

What is output by the code to the right?

- A) 10.5  
B) 24  
C) 23.25  
D) 23  
E) 25

```
int a = 15;
int b = 4;
int c = 3;

int result = a + b * c - a / b;

out.println(result);
```

## Question 3

What is output by the code to the right?

- A) Val: 5 and 3.50  
Path: C:\\temp  
B) Val:  
5 and 3.5  
Path: C:\\temp  
C) Val: 5 and 3.5  
Path: C:\\temp  
D) Val: 5 and 3.5  
Path: C:\\temp

```
int x = 5;
double y = 3.5;
out.print("Val: ");
out.printf("%d and %.1f\\n", x, y);
out.println("Path: C:\\temp");
```

## Question 4

What is output by the code to the right?

- A) hexlo 5  
B) Hello World 11  
C) Hexxo 5  
D) hexxo 5  
E) hello 5

```
String text = " Hello World ";
String result = text.trim().toLowerCase()
    .substring(0, 5).replace('l', 'x');

out.println(result + " " +
    result.length());
```

## Question 5

What is output by the code to the right?

- A) true true false false  
B) false true true false  
C) true true true false  
D) false true false false  
E) false false false false

```
boolean a = true;
boolean b = false;
boolean c = true;

boolean r1 = a && b || c;
boolean r2 = a || b && !c;
boolean r3 = a ^ c;
boolean r4 = !(a && c) || b;

out.println(r1 + " " + r2 + r3 + r4);
```

|                                                                                                                                                                                                                                                                                                                                                                                                                                |                                                                                                                                                                                                                                                                                                                                                                                              |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Question 6</b></p> <p>What is output by the code to the right?</p> <p>A) 9267<br/>B) 8367<br/>C) 8377<br/>D) 826-7</p>                                                                                                                                                                                                                                                                                                   | <pre>double a = Math.floor(8.9); double b = Math.ceil(2.1); long c = Math.round(6.4); int d = Math.abs(-7);  out.println((int)a + "" + (int)b + c + d);</pre>                                                                                                                                                                                                                                |
| <p><b>Question 7</b></p> <p>What is output by the code to the right?</p> <p>A) 89      B) 88      C) 90      D) 91      E) 88</p>                                                                                                                                                                                                                                                                                              | <pre>int a = 26 &amp; 21 + 3; int b = 67   420 / 69; a ^= b - 13 / 2; out.println(a);</pre>                                                                                                                                                                                                                                                                                                  |
| <p><b>Question 8</b></p> <p>What is output by the code to the right?</p> <p>A) B<br/>B) B1<br/>C) B12<br/>D) C2<br/>E) A12</p>                                                                                                                                                                                                                                                                                                 | <pre>int x = 7; int y = 3; String result = "";  if (x &gt; 5)     if (y &gt; 5)         result = "A";     else         result = "B"; else if (x &gt; 3)     result = "C"; else     result = "D";  switch (result) {     case "A":     case "B":         result += "1";     case "C":         result += "2";         break;     default:         result += "3"; }  out.println(result);</pre> |
| <p><b>Question 9</b></p> <p>On which line of code to the right marked by comments should we place the following loop to center the output pyramid?</p> <div data-bbox="110 1591 799 1717" style="border: 1px solid black; padding: 5px; margin: 10px 0;"> <pre>for(int j = 4; j &gt; i; j--) {     out.print(" "); }</pre> </div> <p>A) line 1                      B) line 2<br/>C) line 3                      D) line 4</p> | <pre>for (int i=1; i&lt;=4;i++) {     //line 1     for (int k=1; k&lt;=2*i-1;k++) {         //line 2         out.print("*");         // line 3     }     // line 4     out.println(); }</pre>                                                                                                                                                                                                |

|                                                                                                                                                                                                                                                                                                                                                                                                                           |                                                                                                                                                                                                                                                      |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Question 10</b></p> <p>What is output by the code to the right?</p> <p>A) 10 6 21<br/>B) 6 4 15<br/>C) 10 4 14<br/>D) 10 4 19</p>                                                                                                                                                                                                                                                                                   | <pre>int[] arr = {2, 4, 6, 8, 10};  arr[1] = arr[0] + arr[3]; arr[4] = arr[1] - arr[2];  int result = arr[1] + arr[4] + arr.length;  out.println(arr[1] + " " + arr[4] + " " +     result);</pre>                                                    |
| <p><b>Question 11</b></p> <p>Given a file "data.txt" containing the below data. What is output by the code to the right?</p> <div data-bbox="110 537 797 604" style="border: 1px solid black; padding: 5px; margin: 10px 0;">         Hello World<br/>25 3.75       </div> <p>A) Hello 25 28<br/>B) Hello World 25 28<br/>C) Hello 25 29<br/>D) World 25 28</p>                                                           | <pre>Scanner file = new Scanner(     new File("data.txt"));  String word = file.next(); file.nextLine(); int num = file.nextInt(); double dec = file.nextDouble();  file.close();  out.println(word + " " + num + " " +     (num + (int)dec));</pre> |
| <p><b>Question 12</b></p> <p>What is output by the code to the right?</p> <p>A) 15 12<br/>B) 12 15<br/>C) 12 8<br/>D) 20 15<br/>E) 6 15</p>                                                                                                                                                                                                                                                                               | <pre>int[] nums = {3, 5, 2, 4, 6}; int sum = 0; int product = 1; for (int n : nums) {     if (n % 2 == 0) {         sum += n;     } else {         product *= n;     } } out.println(sum + " " + product);</pre>                                     |
| <p><b>Question 13</b></p> <p>What is output by the code to the right?</p> <p>A) 12 3 5 1<br/>B) 10 3 5 2<br/>C) 13 3 5 1<br/>D) 14 3 5 1</p>                                                                                                                                                                                                                                                                              | <pre>int a = 3; int b = 4; int c = 2; int result = a + ++b * c-- - b % c; out.println(result + " " + a + " " +     b + " " + c);</pre>                                                                                                               |
| <p><b>Question 14</b></p> <p>Which of the following statements about Java primitive data types is TRUE?</p> <p>A) A byte can store values from -128 to 128.<br/>B) When a short with value 32767 is incremented, it becomes 32768.<br/>C) A long and a double both use 32 bits of memory.<br/>D) When a byte with value -128 is decremented, it becomes 127.<br/>E) An int can store any value that a long can store.</p> |                                                                                                                                                                                                                                                      |

|                                                                                                                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                        |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Question 15</b></p> <p>What is output by the code to the right?</p> <p>A) 15 3<br/>B) 20 3<br/>C) 15 4<br/>D) 30 3<br/>E) There is no output due to a runtime error.</p>                                                                                                                                                                      | <pre>ArrayList&lt;Integer&gt; list =     new ArrayList&lt;&gt;();  list.add(10); list.add(20); list.add(30); list.add(1, 15); list.remove(2); list.set(0, 5);  out.println(list.get(1) + " " +     list.size());</pre> |
| <p><b>Question 16</b></p> <p>What is output by the code to the right given that the file a.txt does not exist?</p> <p>A) 23<br/>B) 13<br/>C) There is no output due to a compile error.<br/>D) There is output, followed by a runtime error.<br/>E) There is no output due to a runtime error.</p>                                                  | <pre>try {     String i = "a.txt";     File f = new File(i);     Scanner s = new Scanner(i);     s.nextLine();     out.print(1); } catch (IOException e) {     out.print(2); } finally {     out.print(3); }</pre>     |
| <p><b>Question 17</b></p> <p>What could replace &lt;1*&gt; in the code to the right so that it will compile and run without error (removing items).</p> <p>A) pop                      B) remove                      C) poll<br/>D) A and B.                      E) All of the above.</p>                                                         | <pre>LinkedHashSet&lt;Integer&gt; struct; struct = new LinkedHashSet&lt;&gt;();</pre>                                                                                                                                  |
| <p><b>Question 18</b></p> <p>Assuming &lt;1*&gt; is filled in correctly, what is output by the line marked //q18 code to the right?</p> <p>A) [1, 189, 3]                      B) [189, 3]<br/>C) Output cannot be determined until runtime<br/>D) There is no output due to a compile error.<br/>E) There is no output due to a runtime error.</p> | <pre>struct.add(1); struct.add(2); struct.add(1); struct.add(189); struct.add(3); struct.&lt;1*&gt;(2); struct.&lt;1*&gt;(1);</pre>                                                                                    |
| <p><b>Question 19</b></p> <p>Assuming &lt;1*&gt; is filled in correctly, what is output by the line marked //q19 code to the right?</p> <p>A) 189                      B) true                      C) false<br/>D) There is no output due to a compile error<br/>E) There is no output due to a runtime error.</p>                                 | <pre>out.println(struct); //q18 out.println(struct.remove(0)); //q19</pre>                                                                                                                                             |
| <p><b>Question 20</b></p> <p>Which of the lines in the code to the right first causes an error?</p> <p>A) //1<br/>B) //2<br/>C) //3<br/>D) //4<br/>E) None of the above lines cause any errors.</p>                                                                                                                                                 | <pre>double g = Math.ceil(6); //1 int i = Math.min(g, 2); //2 i = Math.round(g); //3 g = Math.log(i); //4</pre>                                                                                                        |

**Question 21**

What can replace **<1\*>** in the code to the right so that the `wings` instance variable is assigned value 2 and compiles without error?

- A) `static`                      B) `static final`
- C) `final`                        D) Nothing is required.
- E) B or C.                        F) Any of the above.

**Question 22**

Which of the following describes the scope of the `city` instance variable in the `Pigeon` class?

- A) More restrictive than `private`.
- B) Less restrictive than `private`; more than `protected`.
- C) Less restrictive than `protected`; more than `public`.
- D) Less restrictive than `public`.

**Question 23**

What can replace **<2\*>** in the code to the right so that the `fly()` method returns value 2 and the `Bird` interface compiles without warning?

- A) `{return wings;}`            B) `;`
- C) `{return 2;}`                D) A or C.
- E) Any of the above.

**Question 24**

What can replace **<3\*>** in the code to the right so that the `fly()` method returns the value 2?

- A) `super.wings`                B) `wings`
- C) `2`                              D) B or C.
- E) Any of the above.

**Question 25**

Assuming **<1\*>**, **<2\*>**, and **<3\*>** are filled in correctly, what is output by the line marked `//q27` in the code to the right, assuming any errors below line `//q27` are commented out?

- A) `Caw!Caw!Caw!`
- B) `Caw! NYC Caw! LA Caw!`
- C) `Caw! NYC Caw! LA Caw! Miami`
- D) There is no output due to a compile error.
- E) There is no output due to a runtime error.

**Question 26**

Assuming **<1\*>**, **<2\*>**, and **<3\*>** are filled in correctly, and any error from the previous question has been commented out, what is output by the line marked `//q28` in the code to the right?

- A) `0 0 0`                        B) `2 2 2`
- C) `2 0 0`                        D) `2 2 0`
- E) There is no output due to a compile error.

```
interface Bird {
    <1*> int wings = 2;
    public String call();
    private int fly() <2*>
}
```

```
class Pigeon implements Bird {
    String city;
```

```
    public Pigeon(String c) {
        city = c;
    }
```

```
    public String call() {
        return "Caw! " + city;
    }
```

```
    public int fly() {
        return <3*>;
    }
```

```
//////////client code//////////
Pigeon p1 = new Pigeon("NYC");
Pigeon p2 = new Pigeon("LA");
Bird b1 = new Pigeon("Miami");
String o = p1.call();
o += " " + p2.call();
o += " " + b1.call();
out.println(o); //q27
o = "" + p1.fly();
o += " " + p2.fly();
o += " " + b1.fly();
out.println(o); //q28
```



|                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                                                                                                |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Question 27</b></p> <p>What is output if the following call is made to the function <code>mystery</code> to the right? <code>mystery("AHs2");</code></p> <p>A) -102                              B) 108<br/> C) 99                                    D) 117<br/> E) There is no output due to a runtime error.</p>                                                                                                                  | <pre>int mystery(String s) {     if (s.isEmpty())         return 0;     char c = s.charAt(0);     s = s.substring(1);     if (Character.isUpperCase(c))         return 3 * mystery(s);     if (Character.isDigit(c))         return 5 + mystery(s);     if (c &lt; 'l')         return -1 * mystery(s);     return 1 + 2 * mystery(s); }</pre> |
| <p><b>Question 28</b></p> <p>What is output if the following call is made to the function <code>mystery</code> to the right? <code>mystery("K38knd38kOP1");</code></p> <p>A) 210                                  B) -183<br/> C) -237                                D) -234<br/> E) There is no output due to a runtime error.</p>                                                                                                       | <pre>int mystery(String s) {     if (s.isEmpty())         return 0;     char c = s.charAt(0);     s = s.substring(1);     if (Character.isUpperCase(c))         return 3 * mystery(s);     if (Character.isDigit(c))         return 5 + mystery(s);     if (c &lt; 'l')         return -1 * mystery(s);     return 1 + 2 * mystery(s); }</pre> |
| <p><b>Question 29</b></p> <p>What is the intended purpose of method the method <code>goodtime(int i, int j)</code> to the right?</p> <p>A) Return the greatest common divisor of two numbers.<br/> B) Return the least common multiple of two numbers.<br/> C) Return the least prime number between the two.<br/> D) Return the greatest prime number between the two.<br/> E) Return the product of greatest factors of two numbers.</p> | <pre>int goodtime(int i, int j) {     if(i == 0    j == 0)         return 0;     int s = Math.abs(i * j);     while(j != 0) {         int t = i % j;         i = j;         j = t;     }     return s / i; }</pre>                                                                                                                             |
| <p><b>Question 30</b></p> <p>What is output by the client code to the right?</p> <p>A) 1 1                                  B) 216 216<br/> C) 36 126                              D) 6 1<br/> E) There is no output due to a runtime error.</p>                                                                                                                                                                                           | <pre>//////////client code////////// out.print(goodtime(12, 18)); out.print(" "); out.print(goodtime(14, 9));</pre>                                                                                                                                                                                                                            |
| <p><b>Question 31</b></p> <p>What is output by the client code to the right?</p> <p>A) false false                        B) false true<br/> C) true false                         D) true true<br/> E) There is no output due to a compile error.</p>                                                                                                                                                                                     | <pre>String str = "BEEPbeepONtheSTREET"; String reg =     "[A-Z]+[a-z]+* ((\\w\\w)+)"; out.print(str.matches(reg) + " "); reg = "\\w\\D{10,17}...."; out.println(str.matches(reg));</pre>                                                                                                                                                      |
| <p><b>Question 32</b></p> <p>What is output by the code to the right?</p> <p>A) TT                                    B) TF<br/> C) FT                                    D) FF<br/> E) There is no output due to a runtime error.</p>                                                                                                                                                                                                     | <pre>int a = 5; double b = 5.2; out.print((a &gt;= b) ? 'T' : 'F'); out.println((b &lt;= a) ? 'T' : 'F');</pre>                                                                                                                                                                                                                                |
| <p><b>Question 33</b></p> <p>Which of the following ranges accurately describes the list of possible integers that <code>rand</code> can be?</p> <p>A) [5,18]                                B) [5,18]<br/> C) [5,13]                                D) [5,13]<br/> E) None of the above.</p>                                                                                                                                              | <pre>int rand = (Math.random() * 13) + 5;</pre>                                                                                                                                                                                                                                                                                                |

**Question 34**

What is output by the line marked //Q34 in the code to the right?

- A) 1                      B) 4                      C) 5
- D) There is no output due to a runtime error as the `byte[]` entity does not implement `hashCode()`, which is required for all types that are stored in a `HashSet`.
- E) There is no output due to a compile time error as the `byte` arrays are being filled with `char`'s, not `byte`'s.

**Question 35**

Disregarding the output produced by the line marked //Q34 in the code to the right, what is output by the code to the right?

- A) [97, 98, 99, 100]      B) abcd  
[100, 97, 98, 99]      bcda  
[99, 100, 97, 98]      cdab  
[97, 98, 99, 100]      dabc  
[98, 99, 100, 97]      abcd
- C) [a, b, c, d]  
[b, c, d, a]  
[c, d, a, b]  
[d, a, b, c]
- D) It is impossible to know the output until runtime as the output will be dependent on the memory addresses of the arrays, since arrays use the identity hash code.
- E) It is impossible to know the output until runtime due to the nature of it being a `HashSet` and storing arrays; however, the output would have been a permutation of the lines from one of either option A, B, or C.
- F) There is still no output due to an error.

**Question 36**

Consider if `uniques` was instead storing `char[]`'s, each newly created array added to `uniques` was instead of type `char[]`, and `unique` was of type `char[]` in the for loop. Then, what is output by the line marked //Q34 in the code to the right?

- A) 1                      B) 4                      C) 5
- D) There is no output due to a runtime error as the `char[]` entity does not implement `hashCode()`, which is required for all types that are stored in a `HashSet`.
- E) There is no output due to a compile time error.

**Question 37**

Using the same situation described by Question 36 above, and disregarding the output produced by the line marked //Q34, what is output by the code to the right?

- A) The same answer as option A from question 35.      B) The same answer as option B from question 35.
- C) The same answer as option C from question 35.      D) The same answer as option D from question 35.
- E) The same answer as option E from question 35.      F) The same answer as option F from question 35.

```
HashSet<byte[]> uniques =
    new HashSet<byte[]>();
uniques.add(new byte[] {
    'a', 'b', 'c', 'd'
});
uniques.add(new byte[] {
    'b', 'c', 'd', 'a'
});
uniques.add(new byte[] {
    'c', 'd', 'a', 'b'
});
uniques.add(new byte[] {
    'd', 'a', 'b', 'c'
});
uniques.add(new byte[] {
    'a', 'b', 'c', 'd'
});
```

```
out.println(uniques.size()); //Q34
for (byte[] unique : uniques) {
    out.println(unique);
}
```

|                                                                                                                                                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Question 38</b></p> <p>What is output by the code to the right?</p> <p>A) T1T1T1T1T1T1T1T1T1T1T2T2T2T2T2T2T2T2T2T2</p> <p>B) T2T2T2T2T2T2T2T2T2T2T2T1T1T1T1T1T1T1T1T1T1</p> <p>C) T1T2T1T2T1T2T1T2T1T2T1T2T1T2T1T2T1T2T1T2</p> <p>D) T2T1T2T1T2T1T2T1T2T1T2T1T2T1T2T1T2T1T2T1</p> <p>E) It is impossible to determine the output until runtime.</p> <p>F) There is no output due to an error.</p> | <pre>final Lock lock = new ReentrantLock(); StringBuilder sb = new StringBuilder();  Thread t1 = new Thread(() -&gt; {     for (int i = 0; i &lt; 10; i++) {         lock.lock();         sb.append("T1");         lock.unlock();     } });  Thread t2 = new Thread(() -&gt; {     for (int i = 0; i &lt; 10; i++) {         lock.lock();         sb.append("T2");         lock.unlock();     } });  t1.start(); t2.start(); try {     t1.join();     t2.join(); } catch (InterruptedException e) {     e.printStackTrace(); }  out.println(sb);</pre> |
| <p><b>Question 39</b></p> <p>What is the <b>height</b> of the Binary Search Tree (BST) of integers resulting from inserting the elements to the right in order? Write your answer in the blank provided on your answer sheet for this question.</p>                                                                                                                                                     | <pre>First: 59         56         42         16         72</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <p><b>Question 40</b></p> <p>What is the <b>maximum width</b> of the Binary Search Tree (BST) of integers resulting from inserting the elements to the right in order? Write your answer in the blank provided on your answer sheet for this question.</p>                                                                                                                                              | <pre>        83         77         58       v 32 Last: 73</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |

# ★ ANSWER KEY – CONFIDENTIAL ★

## UIL COMPUTER SCIENCE – 2025-2026 DISTRICT

Questions (+6 points for each correct answer, -2 points for each incorrect answer)

- |                  |                  |                  |                       |
|------------------|------------------|------------------|-----------------------|
| 1) <u>  E  </u>  | 11) <u>  A  </u> | 21) <u>  F  </u> | 31) <u>  B  </u>      |
| 2) <u>  B  </u>  | 12) <u>  B  </u> | 22) <u>  B  </u> | 32) <u>  D  </u>      |
| 3) <u>  C  </u>  | 13) <u>  C  </u> | 23) <u>  D  </u> | 33) <u>  E  </u>      |
| 4) <u>  D  </u>  | 14) <u>  D  </u> | 24) <u>  D  </u> | 34) <u>  C  </u>      |
| 5) <u>  A  </u>  | 15) <u>  A  </u> | 25) <u>  C  </u> | 35) <u>  D  </u>      |
| 6) <u>  B  </u>  | 16) <u>  A  </u> | 26) <u>  E  </u> | 36) <u>  C  </u>      |
| 7) <u>  A  </u>  | 17) <u>  B  </u> | 27) <u>  D  </u> | 37) <u>  E  </u>      |
| 8) <u>  C  </u>  | 18) <u>  B  </u> | 28) <u>  C  </u> | 38) <u>  E  </u>      |
| 9) <u>  A  </u>  | 19) <u>  C  </u> | 29) <u>  B  </u> | *39) <u>    4    </u> |
| 10) <u>  D  </u> | 20) <u>  B  </u> | 30) <u>  C  </u> | *40) <u>   12   </u>  |

\* See "Explanation" section below for alternate, acceptable answers.

**Note:** Correct responses are based on **Java SE Development Kit 22 (JDK 22)** from Sun Microsystems, Inc. All provided code segments are intended to be syntactically correct, unless otherwise stated (e.g., "error" is an answer choice) and any necessary Java SE 22 Standard Packages have been imported. Ignore any typographical errors and assume any undefined variables are defined as used.

Explanations:

|     |   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-----|---|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.  | E | $456_8 = 302_{10}$ . $711_8 = 457_{10}$ . $302_{10} + 457_{10} = 759_{10} = 1367_8$ .                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 2.  | B | Simple order of operations.                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 3.  | C | Simple printing with escape sequences and <code>printf</code> .                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| 4.  | D | <code>trim()</code> removes leading/trailing spaces. <code>toLowerCase()</code> makes the string lowercase. Substring makes the string "hello". <code>replace()</code> replaces all occurrences of the first character with the second, making the string "hexxo". The string is 5 characters long.                                                                                                                                                                                                      |
| 5.  | A | Simple boolean logic.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 6.  | B | <code>Math.floor(8.9) → 8.0</code> (rounds down to nearest integer)<br><code>Math.ceil(2.1) → 3.0</code> (rounds up to nearest integer)<br><code>Math.round(6.4) → 6</code> (rounds to nearest, 0.4 rounds down)<br><code>Math.abs(-7) → 7</code> (absolute value)                                                                                                                                                                                                                                       |
| 7.  | A | Order of operations.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 8.  | C | First, the if-else chain: <code>x &gt; 5</code> is true ( <code>7 &gt; 5</code> ), so we enter the inner if. <code>y &gt; 5</code> is false ( <code>3 &gt; 5</code> ), so <code>result = "B"</code> . Then, the switch: <code>result</code> is "B", matching case "B". It assigns <code>result += "1"</code> making it "B1". There's no <code>break</code> , so fall-through occurs to case "C", which assigns <code>result += "2"</code> making it "B12". The <code>break</code> then exits the switch. |
| 9.  | A | The spaces need to be printed before the star in order to center the higher levels of the pyramid with the bottom level.                                                                                                                                                                                                                                                                                                                                                                                 |
| 10. | D | <code>arr[1]</code> becomes $2 + 8 = 10$ , then <code>arr[4]</code> becomes $10 - 6 = 4$ , and result is $10 + 4 + 5 = 19$ .                                                                                                                                                                                                                                                                                                                                                                             |
| 11. | A | <code>file.next()</code> reads "Hello", <code>file.nextLine()</code> consumes the rest of line one (" World"), <code>file.nextInt()</code> reads 25, <code>file.nextDouble()</code> reads 3.75, and <code>(int)dec</code> truncates to 3, so <code>num + (int)dec = 25 + 3 = 28</code> .                                                                                                                                                                                                                 |
| 12. | B | Even numbers (2, 4, 6) are added to sum giving $2 + 4 + 6 = 12$ , odd numbers (3, 5) are multiplied into product giving $1 \cdot 3 \cdot 5 = 15$ .                                                                                                                                                                                                                                                                                                                                                       |
| 13. | C | <code>++b</code> increments <code>b</code> to 5 and uses 5, <code>c--</code> uses 2 then decrements <code>c</code> to 1, <code>b % c</code> uses current values $5 \% 1 = 0$ , then $5 * 2 = 10$ , and finally $3 + 10 - 0 = 13$ .                                                                                                                                                                                                                                                                       |
| 14. | D | When a byte at its minimum value -128 is decremented, it underflows and wraps to the maximum value 127; A is false because byte ranges from -128 to 127 (not 128); B is false because 32767 wraps to -32768; C is false because both long and double use 64 bits; E is false because long is 64 bits and can store larger values than the 32-bit int.                                                                                                                                                    |
| 15. | A | Starting with [10, 20, 30], <code>add(1, 15)</code> inserts at index 1 giving [10, 15, 20, 30], <code>remove(2)</code> removes index 2 giving [10, 15, 30], <code>set(0, 5)</code> replaces index 0 giving [5, 15, 30], so <code>get(1)</code> returns 15 and <code>size()</code> returns 3.                                                                                                                                                                                                             |
| 16. | A | An <code>IOException</code> is thrown since the file does not exist, so we enter the <code>catch</code> block and then before terminating, enter the <code>finally</code> block.                                                                                                                                                                                                                                                                                                                         |
| 17. | B | Only <code>remove</code> will work for a <code>LinkedHashSet</code>                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| 18. | B | <code>LinkedHashSet</code> will stay in the same order as added, and <code>remove</code> will take out the occurrence of the integer given (also there are no duplicates in a set, so only one instance of 1).                                                                                                                                                                                                                                                                                           |
| 19. | C | The <code>LinkedHashSet</code> <code>remove</code> method will return <code>false</code> if nothing is removed, but <code>true</code> if something is removed.                                                                                                                                                                                                                                                                                                                                           |
| 20. | B | <code>Math.min(double, double)</code> returns a double, there is no version for <code>Math.min(double, int)</code> . <code>//3</code> also causes an error, <code>Math.round(double)</code> returns a long, but <code>//2</code> happens first.                                                                                                                                                                                                                                                          |
| 21. | F | Instance variables in an interface default as <code>static</code> and <code>final</code> , so you can define these or not.                                                                                                                                                                                                                                                                                                                                                                               |
| 22. | B | Default scope for instance variable is package-protected, which is less restrictive than <code>private</code> , but more restrictive than <code>protected</code> .                                                                                                                                                                                                                                                                                                                                       |
| 23. | D | A method marked <code>private</code> in an interface must be initialized, but since <code>wings</code> is a <code>final</code> variable with value 2, we can send that or the constant 2 as our return.                                                                                                                                                                                                                                                                                                  |
| 24. | D | <code>super.wings</code> causes an error, but <code>wings</code> can be accessed regularly, and it is equal to constant 2, so either can be returned.                                                                                                                                                                                                                                                                                                                                                    |

|     |   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-----|---|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 25. | C | The method works as intended for all 3 instances, returning "Caw! " followed by the given city name from instantiation.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 26. | E | Since the <code>fly()</code> method is <code>private</code> within the <code>interface</code> , and <code>b1</code> is instantiated with <code>Bird</code> class in the eyes of the compiler, the <code>fly()</code> method is out of scope and causes a compile error.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 27. | D | Recursive Tracing.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 28. | C | Recursive Tracing.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 29. | B | The <code>goodtime()</code> function finds the LCM of the two numbers.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| 30. | C | The LCM of 12 and 18 is 36 ( $12 \cdot 3 = 18 \cdot 2 = 36$ ), and the LCM of 14 and 9 is 126 (no common factors to work from, $14 \cdot 9 = 126$ )                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| 31. | B | The first pattern will match either the group of one or more uppercase sequences directly followed by one or more lowercase sequences, any number of times (the whole group), or the group of a string of an even number of word characters, but at least 2, so it does not match. The second pattern will match a string made up of any word character, followed by between 10-17 non digit characters, followed by 4 characters (can be any single characters), so it does match.                                                                                                                                                                                                                                                                                                                             |
| 32. | D | Integers get promoted to doubles in this situation, meaning that the comparison becomes <code>5.0 &gt;= 5.2</code> , which is <code>false</code> . Note that the order in which <code>a</code> and <code>b</code> appear does not influence how the promotion occurs. Thus, both comparisons are <code>false</code> , and 'F' is printed twice on the same line.                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 33. | E | Note that the line assigning a value to <code>rand</code> does not compile. This is because <code>Math.random()</code> returns a type of <code>double</code> , and needs to be explicitly casted down to an integer before it can be stored in <code>rand</code> (of type <code>int</code> ).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 34. | C | Regardless of what array is stored in a <code>HashSet</code> , the value of <code>hashCode()</code> is dependent on the memory address rather than the value being stored. Since five arrays are dynamically allocated via the <code>new</code> keyword, despite two of the arrays storing the same literal values, they will each point to unique memory addresses, and thus there are five elements in the <code>HashSet</code> .                                                                                                                                                                                                                                                                                                                                                                             |
| 35. | D | Objects that don't have an overwritten <code>toString()</code> method print the name of the object, followed by the character '@' followed by the hex representation of their <code>hashCode()</code> value. Arrays are prefaced by one '[' character for each dimension of the array. Lastly, the JVM has special class names encoded for each of the primitive data types since they themselves are not classes. In the case of a <code>byte</code> , this is the character 'B'. In the case of a <code>char</code> , this is the character 'C'.                                                                                                                                                                                                                                                              |
| 36. | C | Even though the data type has changed here, the same rule from question 34 applies.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| 37. | E | The output will actually be a permutation of option choice B from question 35. Note that <code>out.println()</code> has an implementation for <code>println(byte[] arr)</code> which instead prints the <code>String</code> representation of the <code>byte</code> array. This, however, does not change the fact that the order which these lines appear in is not known until runtime due to the rules regarding array's <code>toString()</code> implementation.                                                                                                                                                                                                                                                                                                                                             |
| 38. | E | A common use of a <code>Lock</code> (also known as a mutex which is short for mutual exclusion) is used to ensure that no two processes which use a shared resource modify said shared resource at the same time. Without some additional techniques, it does not, however, determine the order in which said processes access the shared resource. In this case, it is impossible to determine what order the "T1"s and "T2"s will occur in. Rather, we just know that some permutation of ten "T1"s and ten "T2"s will be printed. Thus "T1T2T2T1T2T2T2T1..." is also a valid output that can be generated by this program. This behavior can be more easily observed by instead changing the loop bound from 10 to something like 100 as this is largely dependent on CPU scheduling and instruction cycles. |

|     |    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-----|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 39. | 4  | <p>The nodes when inserted in the order specified result in the following BST:</p> <pre> graph TD     0059((0059)) --&gt; 0056((0056))     0059 --&gt; 0072((0072))     0056 --&gt; 0042((0042))     0056 --&gt; 0058((0058))     0042 --&gt; 0016((0016))     0016 --&gt; 0032((0032))     0072 --&gt; 0083((0083))     0083 --&gt; 0077((0077))     0077 --&gt; 0073((0073)) </pre> <p>The height of a binary tree is the number of edges on the longest path from the root node to the deepest leaf node. Here that value is 4</p>        |
| 40. | 12 | <p>The nodes when inserted in the order specified result in the BST shown above. The maximum width of a binary tree is the maximum of the widths of all levels of the binary tree. The width of a level of a binary tree is the number of nodes between the leftmost and rightmost non-null nodes, inclusive of any null nodes (empty spots) in between. Here, this occurs on the level which contains nodes 32 and 73, which is represented as [32, null, null, null, null, null, null, null, null, null, null, 77] which has width 12.</p> |